

Lab experiment-8

Experiment 8: Depth First Search (DFS)

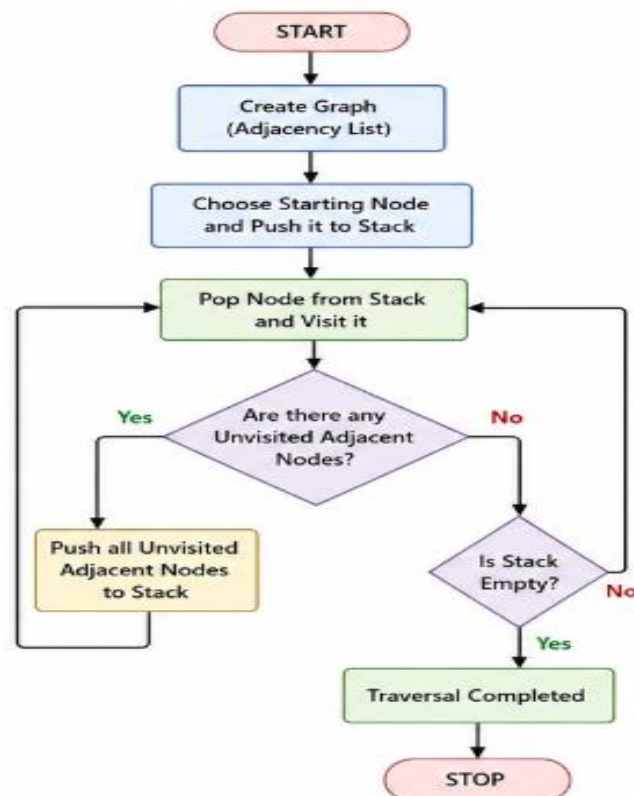
Aim

To implement the Depth First Search (DFS) algorithm using Python.

Objective

To traverse all the nodes of a graph using the DFS algorithm.

Flowchart



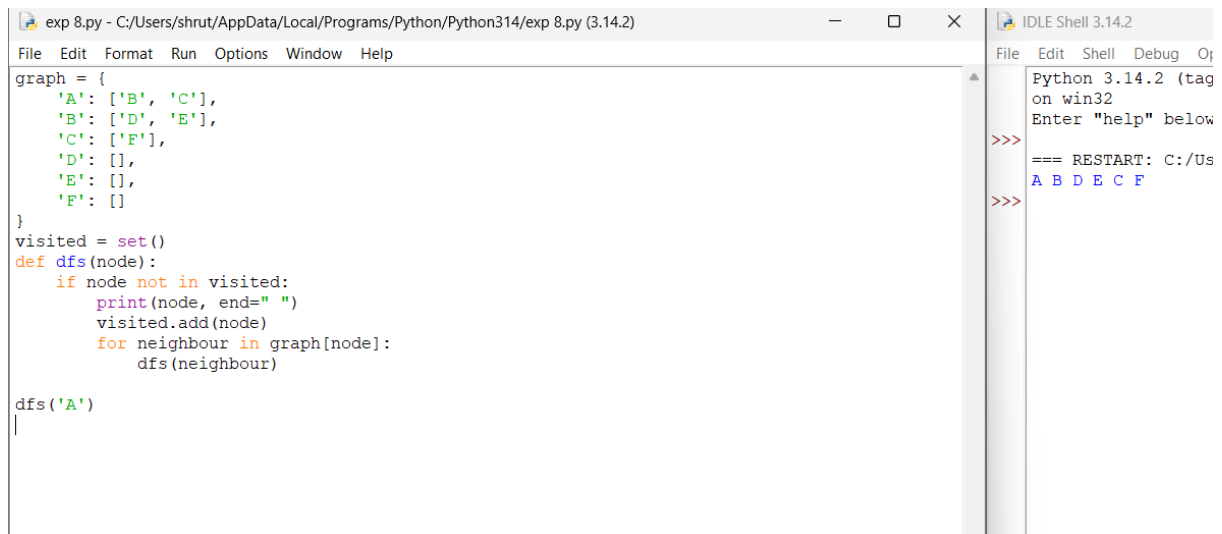
Algorithm

1. Create a graph using an adjacency list.
2. Mark the starting node as visited.
3. Visit the current node.
4. Recursively visit all unvisited adjacent nodes.
5. Continue until all nodes are visited.

Code

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}  
  
visited = set()  
  
def dfs(node):  
    if node not in visited:  
        print(node, end=" ")  
        visited.add(node)  
        for neighbour in graph[node]:  
            dfs(neighbour)  
  
dfs('A')
```

Output



The screenshot shows a Python IDE window titled 'exp 8.py - C:/Users/shrut/AppData/Local/Programs/Python/Python314/exp 8.py (3.14.2)'. The main editor displays the following Python code:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
visited = set()
def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(neighbour)
dfs('A')
```

The right-hand pane shows the IDLE Shell 3.14.2 with the following output:

```
Python 3.14.2 (tag
on win32
Enter "help" below
>>>
=== RESTART: C:/Us
A B D E C F
>>>
```

Result

The graph was successfully traversed using the DFS algorithm.

Conclusion

The DFS algorithm visits nodes by exploring each branch completely before backtracking.